1ª Entrega do Projeto • Programação Orientada a Objetos em Java

Controle de Estoque

Modelagem e Implementação em Java dos Fundamentos de Orientação a Objetos: Encapsulamento, Sobrecarga de Construtores e Associação de Classes

INSTITUIÇÃO & DISCIPLINA
IFRN • Programação Orientada a Objetos

PROFESSORA AVALIADORA
Tatiana Amaral Sorrentino

EQUIPE DE APRESENTAÇÃO (4 INTEGRANTES)

- Gian (Contextualização & Regras)
- Paulo (Modelagem & UML)
- Heitor (Implementação de Domínio)
- Celine (Execução & Validação)

VISÃO GERAL DO PROJETO

O que foi feito? Escopo do Sistema

Objetivo do Trabalho

Construção de uma solução computacional em Java voltada para a gestão de mercadorias físicas e controle do inventário de um estabelecimento comercial.

O foco primordial foi mapear entidades reais de uma cadeia de suprimentos para classes Java bem estruturadas, garantindo integridade de dados e separação estrita de responsabilidades.

Entidades Modeladas

- **Classe Produto:** Modela o item do inventário, armazenando nome descritivo, preço unitário monetário e saldo físico em unidades.
- **Classe Fornecedor:** Modela a distribuidora parceira com nome institucional e telefone de contato.
- **Vínculo Direto:** O produto referencia o fornecedor por associação de objetos, espelhando a relação de suprimento do mundo real.

JUSTIFICATIVA

Por que este tema? Motivação & Aderência a POO

Relevância Prática Comercial

O controle de estoque é o coração operacional de qualquer empresa comercial (papelarias, minimercados, lojas de informática).

Falhas de controle causam rupturas de estoque (perda de clientes por falta de produtos) ou imobilização excessiva de capital financeiro parado nas prateleiras.

Aderência Ideal aos Conceitos de POO

O domínio comercial oferece um ambiente límpido para exercitar as regras da disciplina:

- **Abstração Clara:** Identificação fácil e intuitiva dos atributos essenciais de cada entidade.
- **Encapsulamento Mandatório:** Preços e saldos de mercadoria não podem sofrer alterações desordenadas sem métodos de controle.
- **Associação Natural:** A relação "Produto tem um Fornecedor" evita abstrações forçadas.

REGRAS DO SISTEMA

Regras de Negócio e Funcionamento Geral

Regras de Negócio Estabelecidas

- **Autonomia do Fornecedor:** O fornecedor é instanciado de forma autônoma e pode estar associado a múltiplos produtos cadastrados no catálogo.
- **Integridade do Produto:** O produto mantém o controle quantitativo e financeiro de seu lote.
- **Programação Defensiva:** Tratamento seguro contra valores nulos caso o produto seja consultado antes de possuir fornecedor definido.

Ciclo de Execução do Sistema

1. **Instanciação dos Parceiros:** Criação dos objetos fornecedores em memória.

2. **Alocação dos Produtos:** Registro dos itens via construtor com parâmetros ou montagem gradual via métodos `set`.

3. **Consolidação do Inventário:** Chamada do método `exibirInformacoes()` para cruzamento e impressão no console.

ENGENHARIA DE SOFTWARE

Pilares de POO Aplicados no Trabalho

Encapsulamento Estrito (private)

Todos os atributos das classes de domínio foram declarados estritamente como `private`.

O acesso aos estados é realizado exclusivamente por métodos assessores (`get`) e modificadores (`set`), protegendo os dados contra corrupção externa direta.

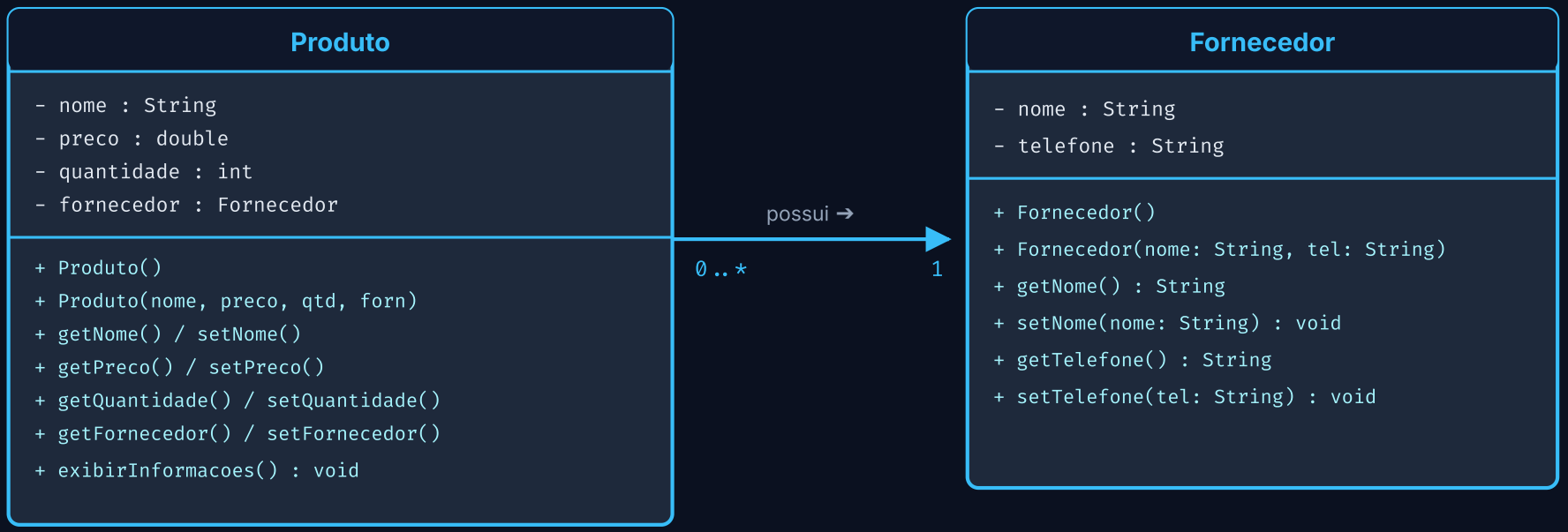
Sobrecarga de Construtores

Implementação dos dois formatos exigidos pelas regras da avaliação:

- **Construtor Padrão:** Inicialização flexível para preenchimento tardio de atributos.
- **Construtor Parametrizado:** Inicialização atômica que entrega a instância em estado consistente em um único comando `new`.

MODELAGEM VISUAL OFICIAL

Diagrama de Classes em Notação UML



CONEXÃO ENTRE ENTIDADES

Relacionamento: Por que Associação de Objetos?

Inconveniente do Modelo Procedural

Se colocássemos atributos `nomeFornecedor` e `telefoneFornecedor` diretamente dentro da classe `Produto`:

- **Redundância Severa:** Se 100 produtos forem da mesma distribuidora, repetiríamos o telefone 100 vezes.
- **Inconsistência de Dados:** Se o telefone do fornecedor mudasse, teríamos que atualizar 100 registros manuais.

Vantagem da Associação em POO

O `Produto` guarda apenas a referência em memória para o objeto `Fornecedor`:

- **Reuso de Memória:** Diversos produtos apontam para a mesma instância `f1`.
- **Coesão & Baixo Acoplamento:** Cada classe evolui de forma limpa e independente.

CÓDIGO-FONTE DAS CLASSES

Classe Fornecedor.java

```java
public class Fornecedor {
    // Atributos protegidos por encapsulamento
    private String nome;
    private String telefone;

    // 1. Construtor Padrão (Sem argumentos)
    public Fornecedor() {
    }

    // 2. Construtor Parametrizado
    public Fornecedor(String nome, String telefone) {
        this.nome = nome;
        this.telefone = telefone;
    }

    // Encapsulamento com Getters e Setters
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }

    public String getTelefone() { return telefone; }
    public void setTelefone(String tel) { this.telefone = tel; }
}
```

Análise Técnica de POO

- **Visibilidade Private:** Garante que os atributos não possam ser acessados nem alterados livremente fora da classe.
- **Sobrecarga:** A presença do construtor padrão e parametrizado atende integralmente às regras da professora.
- **Palavra-chave `this`:** Utilizada para diferenciar o atributo da instância do parâmetro recebido no construtor.

CÓDIGO-FONTE DAS CLASSES

Classe Produto: Atributos e Construtores

```java
public class Produto {
    private String nome;
    private double preco;
    private int quantidade;

    // Associação com a classe Fornecedor
    private Fornecedor fornecedor;

    // Construtor Padrão
    public Produto() {
    }

    // Construtor Parametrizado Completo
    public Produto(String nome, double preco,
                   int quantidade, Fornecedor fornecedor) {
        this.nome = nome;
        this.preco = preco;
        this.quantidade = quantidade;
        this.fornecedor = fornecedor;
    }
    // (Getters e Setters na sequência...)
```

Análise Técnica de POO

- **Atributo Tipo Objeto:** O campo `private Fornecedor fornecedor;` demonstra a composição estrutural entre classes.
- **Injeção no Construtor:** Permite construir um objeto `Produto` já vinculado ao seu parceiro distribuidor.
- **Tipos Primitivos Apropriados:** Uso de `double` para moeda e `int` para contagem física.

CÓDIGO-FONTE DAS CLASSES

Classe Produto: Comportamento & Métodos

```java
public void exibirInformacoes() {
    System.out.println("Produto: " + nome);
    System.out.println("Preço: R$" + preco);
    System.out.println("Quantidade em Estoque: " + quantidade);

    // Tratamento defensivo contra NullPointerException
    if (fornecedor != null) {
        System.out.println("Fornecedor: " + fornecedor.getNome()
            + " | Tel: " + fornecedor.getTelefone());
    } else {
        System.out.println("Fornecedor: Não cadastrado");
    }
    System.out.println("---------------------------------------");
}
```

Análise Técnica de POO

- **Navegação entre Objetos:** Invoca `fornecedor.getNome()` e `fornecedor.getTelefone()`, respeitando o encapsulamento.
- **Segurança de Execução:** A verificação `if (fornecedor != null)` garante robustez caso o produto não tenha fornecedor atribuído inicialmente.
- **Responsabilidade Única:** O método formata apenas dados próprios e solicita dados de terceiros educadamente via getters.

ORQUESTRAÇÃO EXECUTÁVEL

Classe Main: Instanciação e Comparação

```java
public class Main {
    public static void main(String[] args) {
        // 1. Instanciação do Fornecedor
        Fornecedor f1 = new Fornecedor(
            "Distribuidora Silva", "(00) 00000-0000"
        );

        // 2. Instanciação via Construtor Parametrizado
        Produto p1 = new Produto(
            "Caderno 10 matérias", 22.50, 40, f1
        );

        // 3. Instanciação via Construtor Padrão + Setters
        Produto p2 = new Produto();
        p2.setNome("Caneta Esferográfica");
        p2.setPreco(2.00);
        p2.setQuantidade(150);
        p2.setFornecedor(f1); // Reuso da mesma instância f1

        // 4. Exibição
        p1.exibirInformacoes();
        p2.exibirInformacoes();
    }
}
```

Destaques de POO na Prática

- **Exercício dos 2 Construtores:** O produto `p1` valida o construtor completo; o produto `p2` valida o construtor vazio com preenchimento via `set`.
- **Compartilhamento de Referência:** Ambos os produtos apontam para o mesmo objeto `f1` na memória heap, comprovando o reuso de instâncias.

DEMONSTRAÇÃO PRÁTICA

Execução do Programa: Saída do Console

```
=== SISTEMA DE CONTROLE DE ESTOQUE ===

Produto: Caderno 10 matérias
Preço: R$22.5
Quantidade em Estoque: 40
Fornecedor: Distribuidora Silva | Tel: (00) 00000-0000
--------------------------------------
Produto: Caneta Esferográfica
Preço: R$2.0
Quantidade em Estoque: 150
Fornecedor: Distribuidora Silva | Tel: (00) 00000-0000
--------------------------------------
```

Análise da Execução na JVM

- **Integridade Individual:** Os atributos específicos (nome, preço e saldo) permaneceram estritamente independentes em cada objeto.
- **Resolução de Associação:** Os dois produtos buscaram e exibiram os contatos da Distribuidora Silva sem duplicar strings no código.

VALIDAÇÃO ACADÊMICA

Atendimento aos Critérios da Disciplina

CRITÉRIO EXIGIDO PELA PROFª TATIANA (IFRN)	STATUS	EVIDÊNCIA NO TRABALHO
Mínimo de 2 classes + 1 com método main (min. 3 classes)	✓ 100% Atendido	3 classes implementadas: `Produto`, `Fornecedor` e `Main`
Encapsulamento com utilização correta do `private`	✓ 100% Atendido	Todos os atributos de `Produto` e `Fornecedor` são `private`
Utilização dos métodos `gets` e `sets`	✓ 100% Atendido	Getters e Setters completos em todas as classes de domínio
Construtores: Parametrizável e Padrão	✓ 100% Atendido	Ambos implementados nas classes e testados na `Main`
Relacionamento entre Objetos de classes diferentes	✓ 100% Atendido	Associação unidirecional entre `Produto` e `Fornecedor`
Participação de todos do grupo na apresentação	✓ 100% Atendido	Divisão equitativa para Gian, Paulo, Heitor e Celine (~5 min cada)

ENCERRAMENTO OFICIAL

Conclusão e Agradecimentos

Conclusões da Equipe

O paradigma de Orientação a Objetos proporcionou:

- Organização de software muito superior ao paradigma puramente estruturado.
- Proteção de integridade por meio de encapsulamento com regras de acesso.
- Arquitetura pronta para herança e polimorfismo nas próximas etapas.

Muito Obrigado!

Agradecemos à **Professora Tatiana Sorrentino** e aos colegas de turma.

EQUIPE DE DESENVOLVIMENTO

Gian • Paulo • Heitor • Celine

IFRN • POO 2026

1ª Entrega Concluída